# [CSE299(2) - Group 03] Weekly Report: AI Diff Checker

**Name :** Md Sanaul Islam Zihad
**Student ID :** 2311774042
**Email :** sanaul.zihad@northsouth.edu

## I. Weekly Report 1

### A. *Follow-up from last week*

No previous tasks assigned.

### B. *Work narrative of the current week*

- **Tasks completed:**
  - At the beginning of this week, we decided to build our frontend using React since our project requires a modern and interactive user interface and it can easily communicate with our Python backend later on. After deciding this, we set up the React environment and started working with its basic structure [7].

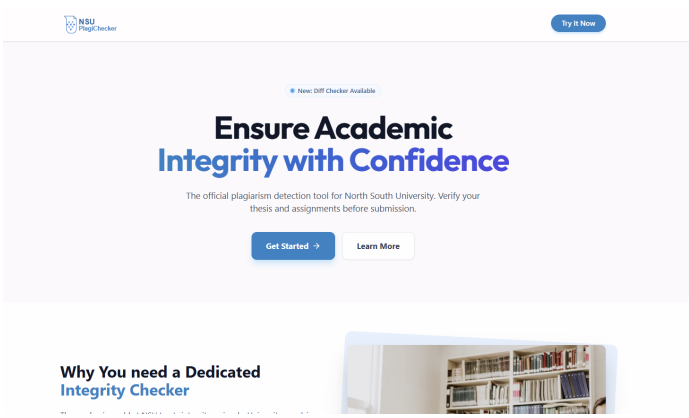


Fig. 1: Frontend Landing Page Design

  - After getting familiar with React, we designed a landing page for our plagiarism detection system. The landing page includes the project overview, system purpose, and navigation buttons so users can easily understand how the system works.
  - Next, we worked on designing a submission page where users can upload their documents for plagiarism checking. Currently, the "Try Now" button on the landing page takes the user to the checking interface.

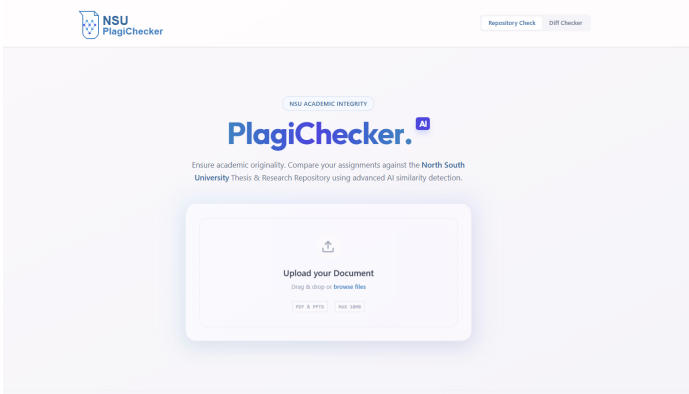


Fig. 2: Repository Check for large dataset plagiarism check

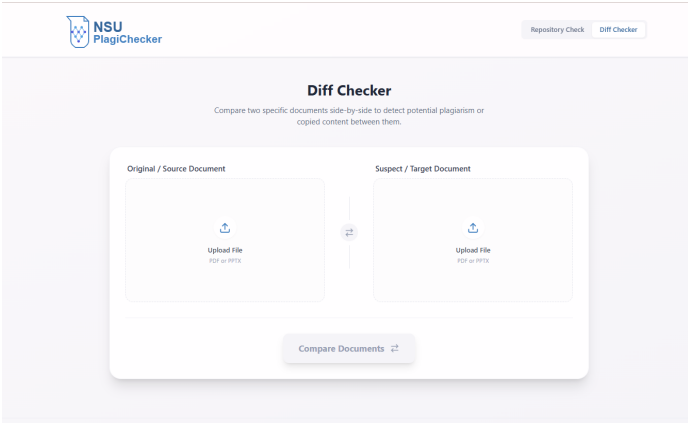


Fig. 3: Diff Checker for one-to-one PDF comparison

- Inside the system, we planned two separate checking containers. The first one is the Repository Check, which is designed to compare one document against multiple reference files for large dataset plagiarism checking. The second one is the Diff Checker, which is designed for one-to-one PDF comparison.
- Since this project is AI-based, we also started researching semantic similarity and vector embeddings to understand how plagiarism can be detected even when sentences are paraphrased [5,8].
- Furthermore, we researched pretrained models under the Sentence-Transformers library to find a suitable model for similarity detection [11].
- To maintain collaboration and version control, we created a GitHub repository and added group members as collaborators.
- We also started designing the overall system architecture including repository indexing and document checking workflow.

- **Challenges faced:**
  - During the research phase, we faced several challenges while trying to understand how similarity detection actually works in plagiarism checking systems. One major difficulty was understanding how systems detect similarity when content is paraphrased. It was confusing to understand how AI models calculate similarity scores and how the percentage of similarity is determined between two texts.
  - Another challenge was researching how matched text can be highlighted inside PDF documents. Understanding how to locate the exact matching portion of text and visually highlight it inside the original document required additional research and experimentation.
  - We also faced difficulty while researching how to keep the system running locally instead of using cloud-based services. Since cloud solutions are commonly used for AI processing, finding suitable methods to implement document processing, vector storage, and similarity checking locally required extra study and testing.
  - To overcome these challenges, we followed online tutorials, technical documentation, and discussion forums which helped us understand these concepts gradually.
- **Progress made:**
  During this week, we successfully built the initial frontend layout including the landing page and submission interface. We also established our project collaboration environment through GitHub. In addition, we gained a basic understanding of semantic similarity techniques and finalized the initial architecture design of our plagiarism detection system. This gives us a strong foundation to start backend development in the upcoming weeks.

### C. *Target for next weekly update*

1) In the next week, we plan to start implementing the text processing pipeline of our system. Our main goal will be to extract text from uploaded PDF and PPTX files and prepare the text for similarity checking.
2) We will research and finalize a method for PDF text extraction using libraries like PyPDF2 or pdfplumber. We plan to select one stable method and create a helper function that can extract text page by page.
3) We will also implement PPTX text extraction using the python-pptx library so that slide contents can be collected and processed properly.
4) After extracting the text, we will work on text cleaning techniques such as removing extra spaces, unwanted newline characters, and unnecessary special characters to make the text suitable for processing.
5) We will then design and implement a chunking strategy where large documents will be divided into smaller segments based on token or word limits. We may also experiment with overlapping chunks to improve similarity detection accuracy.

6) Additionally, we plan to design a metadata structure to store document information such as document ID, file name, file path, number of chunks, and indexing time. This will help us organize and manage repository documents efficiently.

## II. Weekly Report 2

### A. *Follow-up from last week*

1) Done
2) Done
3) Done
4) Done
5) Done
6) Done

### B. *Work narrative of the current week*

- **Tasks completed:**
  - We worked on improving the frontend by adding a repository feature where teachers can upload their own documents to a database, and these files will later be used for comparison [4]. We also added a ”Compare against” option where teachers can choose whether to match the uploaded file with the whole university repository or their personal repository. Additionally, any uploaded files are now stored and shown in a ”Past Repository” section so teachers can view and manage their previous uploads.

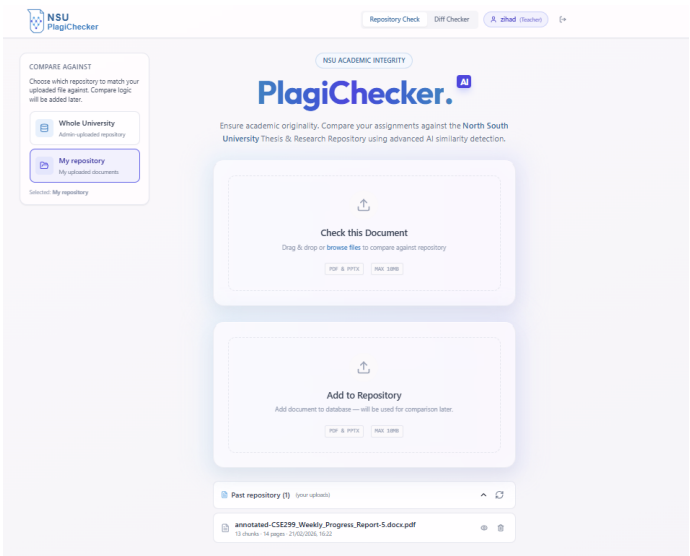


Fig. 4: Teacher Landing Page Design

  - We completed several important implementation tasks to strengthen our system pipeline. We successfully implemented PDF text extraction using PyPDF2 and pdfplumber [10,13], allowing us to extract content page by page. Alongside this, we also developed PPTX text extraction using the python-pptx library [1], enabling us to collect slide-wise content efficiently. After extraction, we worked on text cleaning, where we removed unnecessary spaces, newline characters, and unwanted control symbols to prepare the text for further processing. We then designed and applied a chunking strategy, dividing large documents into smaller segments based on word limits, while also introducing overlap between chunks to improve similarity detection. Finally, we structured a metadata system to organize our documents.

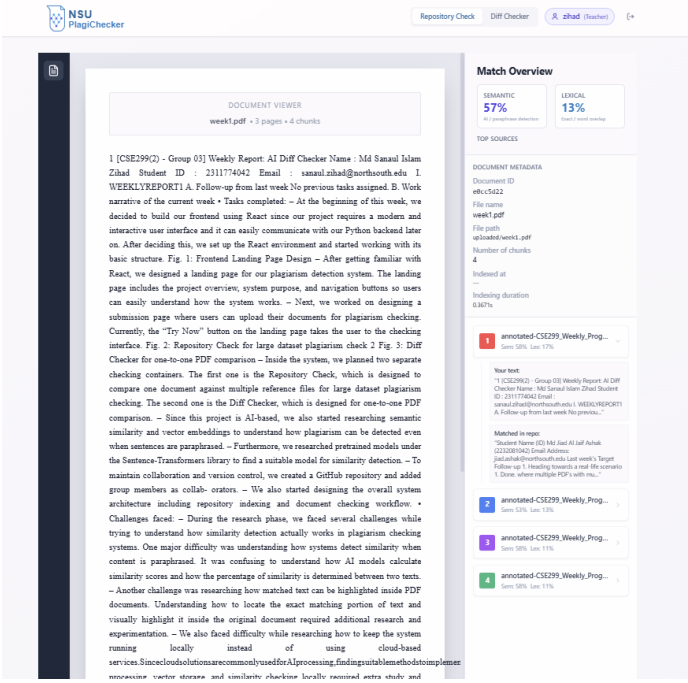


Fig. 5: PDF text extraction and similarity detection

- This week, we also successfully implemented a semantic similarity model as a core part of our system. We integrated the Sentence-Transformers model `paraphrase-MiniLM-L6-v2` [11] to generate meaningful text embeddings. Using this model, we developed an `encode_chunks()` function to convert processed text chunks into vector representations. For similarity measurement, we applied cosine similarity [12]. In addition, we designed a matching mechanism using a `find_matches()` function and stored embeddings in a dedicated `document_chunk_embeddings` table for efficient retrieval.

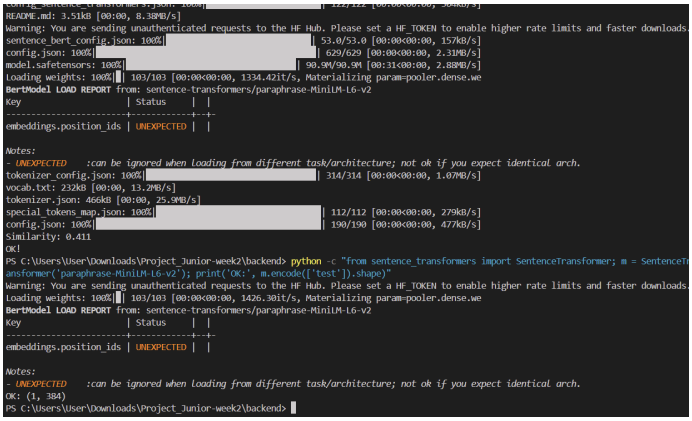


Fig. 6: Semantic similarity model score

- **Challenges faced:**
  - One of the main challenges we faced was implementing the comparison logic between different repositories. Specifically, handling the option where teachers can choose to compare a document either with the whole university repository or only with their personal repository was somewhat complex. Managing this selection dynamically and ensuring the correct dataset is used during comparison required careful design and testing.
  - Another challenge was related to text extraction and preprocessing. The extracted text was not always clean or fully accurate, especially for complex PDF and PPTX files. Additionally, our current chunking strategy (e.g., 200-word chunks with overlap) sometimes caused incomplete context during comparison, which led to inaccurate similarity results. We need to further refine the extraction and chunking process to ensure better consistency and coverage of the full text.
  - We also observed that the AI model is not yet producing highly accurate similarity results in all cases. Although the implementation is functional, the quality of matching needs improvement. To address this, we plan to experiment

with more advanced or alternative models to achieve better semantic understanding and improve overall detection performance.

- **Progress made:**
  During this week, we made significant progress in both frontend and backend development of our system. We enhanced the frontend by introducing a repository feature that allows teachers to upload and manage their documents, along with a flexible comparison option between university and personal repositories. On the backend side, we successfully implemented the complete text processing pipeline, including PDF and PPTX text extraction, text cleaning, chunking, and metadata structuring. Furthermore, we integrated a semantic similarity model using Sentence-Transformers and established the embedding generation, storage, and retrieval workflow. These developments have moved our project from initial design to a functional prototype, setting a strong foundation for improving accuracy and expanding system capabilities in the upcoming weeks.

### C. *Target for next weekly update*

1) In the next week, our primary goal will be to implement the `/compare` endpoint along with completing the student PDF workflow. In the previous week, we completed the teacher-side comparison, and this week we aim to implement a similar workflow for students so that uploaded PDFs can be compared properly with similarity scores and matched outputs.
2) We plan to integrate FAISS vector indexing to replace the current brute-force similarity search. This will significantly improve the efficiency and speed of our system when handling large document repositories.
3) After setting up FAISS, we will implement FAISS-based Top-K retrieval so that the system can efficiently return the most relevant matching chunks without relying on slow looping methods.
4) Additionally, we aim to introduce an N-gram fingerprint verification layer to strengthen our detection system and improve accuracy by combining it with semantic similarity.
5) If time permits, we will further research and experiment with improved models to achieve better similarity results and overall system performance.
6) We may also start working on a conventional diff checking method (such as Myers diff) to show line-by-line or character-level differences between documents.
7) Furthermore, we plan to explore a highlighting system so that matched text portions can be visually marked inside documents for better user understanding.

## III. Weekly Report 3

### A. *Follow-up from last week*

1) Done
2) Done
3) Done
4) Done
5) Done
6) Done
7) Partially Done

### B. *Work narrative of the current week*

- **Tasks completed:**
  - We improved the frontend by updating the Check Document section and adding more repository management features. In this section, we introduced three comparison options so that users can choose which repository they want to compare their uploaded document against: University Repository, My Repository, and Both. Depending on the selected option, the system will match the document with the selected dataset. We also implemented an admin repository upload feature where administrators can upload documents to the university repository. Additionally, the home page UI was updated to improve the overall user interface and navigation.

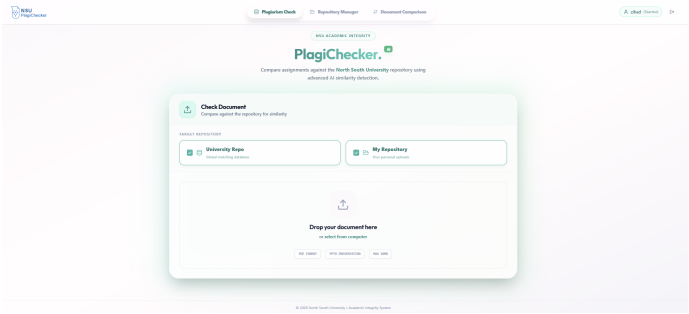


Fig. 7: Update Check Document section

  - We began integrating FAISS vector indexing to replace the current brute-force similarity search, which will significantly improve the efficiency and speed of the system when working with large document repositories [3].
  - After setting up FAISS, we started implementing FAISS-based Top-K retrieval so that the system can efficiently return the most relevant matching chunks without relying on slow looping methods [6].
  - We also started working on adding an N-gram fingerprint verification layer to strengthen the detection system and improve accuracy by combining lexical matching with semantic similarity [2].
  - We continued researching improved embedding models that may provide better similarity results and overall system performance [11].
  - Finally, we explored implementing a traditional diff checking method (such as Myers diff) so that the system can show line-by-line or character-level differences between documents [9].
- **Challenges faced:**
  - During this week, we faced several challenges while testing and improving the system.
  - One of the main problems was with the similarity detection model. In some cases, the model was unable to detect similarity between documents even when there were clearly similar sentences. Sometimes even slightly paraphrased text was not detected, which made us realize that we need to further study how similarity scores and thresholds should be set for better plagiarism detection.
  - Another issue appeared when working with large PDF documents. The chunking system did not work properly for bigger files and in some cases it generated only a small number of chunks instead of dividing the document correctly. Because of this, some parts of the document were not included in the comparison process.
  - We also faced problems with the highlighting system. Currently, highlighting works correctly on cleaned text, but when we try to map the highlighted portion back to the original document, the highlighted area sometimes becomes incorrect. This happens because chunking and text preprocessing change the original text positions.
  - Another challenge was deciding how to properly calculate similarity scores in the backend. We are still researching which scoring method should be used for plagiarism detection, especially for paraphrased content, and how the similarity percentage should be determined.
- **Progress made:**
  Despite facing several challenges, we were able to make steady progress in improving the system. The frontend features for repository selection and document comparison were successfully updated, and the overall interface became more structured and user-friendly. We also started integrating FAISS for faster similarity search and continued experimenting with chunking and similarity detection methods. Although some issues still remain with chunking and highlighting, we now have a clearer understanding of how the system components should work together, which will help us improve the backend logic in the next stage.

### C. *Target for next weekly update*

1) We plan to improve the plagiarized text highlighting logic so that matched text portions can be accurately identified and marked inside the original document.
2) We will work on improving the report visualization format, where the document structure remains unchanged but the detected plagiarized text will be highlighted clearly for better understanding.
3) We aim to further improve the FAISS search, text cleaning process, and N-gram verification layer so that the similarity detection becomes more accurate and efficient.
4) We will also design a similarity threshold classification plan to determine how similarity percentages should be interpreted for plagiarism detection.

# IV. Weekly Report 4

# V. Weekly Report 5

# References


[1] S. Canny. python-pptx documentation, 2024.
[2] W. Cavnar and J. Trenkle. N-gram based text similarity detection. *Computational Linguistics*, 1994.
[3] J. Johnson, M. Douze, and H. Jegou. Faiss: A library for efficient similarity search. *Facebook AI Research*, 2017.
[4] M. Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
[5] Y. e. a. Li. A survey on semantic similarity in nlp. *IEEE TKDE*, 2006.
[6] Y. Malkov and D. Yashunin. Approximate nearest neighbor search for high dimensional vectors. *IEEE*, 2018.
[7] Meta Open Source. React documentation, 2024.
[8] T. e. a. Mikolov. Distributed representations of words and phrases. *NeurIPS*, 2013.
[9] E. Myers. An o(nd) difference algorithm and its variations. *Algorithmica*, 1986.
[10] PyPDF2 Developers. Pypdf2 documentation, 2024.
[11] N. Reimers and I. Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *EMNLP*, 2019.
[12] G. Salton and M. McGill. Vector space model for information retrieval. *Communications of the ACM*, 1983.
[13] J. V. and Contributors. pdfplumber documentation, 2024.